Part 2: Guitar37 Class

In the second part of the assignment, you are going to build on the GuitarString class to write a class that keeps track of a musical instrument with multiple strings. There could be many possible guitar objects with different kinds of strings. As a result, we introduce an interface known as Guitar that each guitar object implements.

The Guitar interface is defined as follows:

```
public interface Guitar {
    public void playNote(int pitch);
    public boolean hasString(char key);
    public void pluck(char key);
    public double sample();
    public void tic();
    public int time();
}
```

The interface allows a client to specify what to play in one of two ways. A client can specify exactly which note to play by calling the playNote method passing it a pitch. Pitch is specified as an integer where the value 0 represents concert-A and all other notes are specified relative to concert-A using what is known as a chromatic scale. Not every value of pitch can be played by any given guitar. If it can't be played, it is ignored.

A client can also specify a character that indicates which note to play by calling the pluck method. Different guitar objects will have different mappings from characters to notes. The interface includes a method called hasString that is paired with pluck that lets a client verify that a particular character has a corresponding string for this guitar. The pluck method has a precondition that the key is legal for this guitar.

The Guitar interface also has methods for getting the current sound sample (the sum of all samples from the strings of the guitar), to advance the time forward one "tic," and an optional method for determining the current time (the number of times tic has been called). If the time method is not implemented, it returns -1.

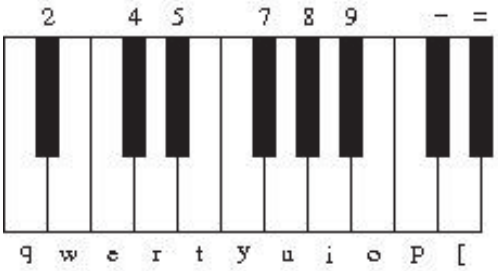
You are being provided with a sample class called GuitarLite that implements the Guitar interface. Once you have verified that your GuitarString class passes the testing program, you can play the GuitarLite instrument. It has only two strings: a and c. Keep in mind that GuitarLite does not have a main method. There is a separate class called GuitarHero that has main (the initial version constructs a GuitarLite object).

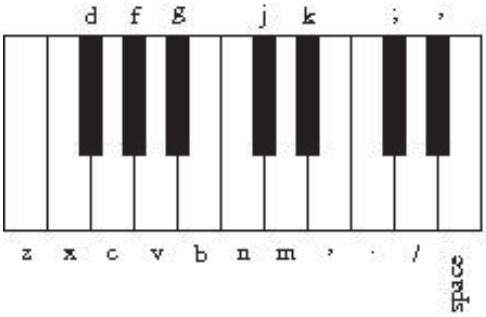
In this second part of the assignment, your task is to make a variation of GuitarLite known as Guitar37. It will model a guitar with 37 different strings. Because it has so many strings, we will want to keep track of them in a data structure. Your Guiter37 objects should each keep track of an array of 37 GuitarString objects.

The Guitar37 class has a total of 37 notes on the chromatic scale from 110Hz to 880Hz. We will use the following string to map keys typed by the user to positions in your array of strings. The i-th character of this string should correspond to the i-th character of your array:

```
"q2we4r5ty7u8i9op-[=zxdcfvgbnjmk,.;/' "
```

This use of keyboard characters imitates a piano keyboard, making playing songs a little easier for people used to a piano keyboard. The white keys are on the qwerty and zxcv rows and the black keys on the 12345 and asdf rows of the keyboard, as in the drawing below.





You are being provided a skeleton version of the Guitar37 class that includes this string defined as a constant called KEYBOARD. The i-th character of the string corresponds to a frequency of $440 \times 2^{(i - 24) / 12}$, so that the character “q” is 110Hz, “i” is 220Hz, “v” is 440Hz, and “ ” (space) is 880Hz.

As noted above, a pitch of 0 is supposed to correspond to concert-A, which will be at index 24 for the Guitar37 object (corresponding to the character “v”). Thus, you can convert from a pitch value to an index in your string by adding 24 to the pitch value. The table below shows some examples of this conversion.

Key	Pitch
"q"	-24
"2"	-23
"w"	-22
"e"	-21
...	...
"v"	0
...	...
"/"	10
"'"	11
" "	12

In working on this second part of the assignment, you are generalizing the code that you will find in GuitarLite. Because that instrument has just two strings, it uses two separate fields. Your instrument has 37 strings, so it uses an array of strings. Each of the operations defined in the interface needs to be generalized from using two specific strings to using an array of strings. For example, the sample method returns the sum of the current samples. GuitarLite does this by adding together two numbers. Your version will have to use a loop to find the sum of all 37 samples.

The GuitarLite class is not well documented and does not handle illegal keys. Your Guitar37 class should include complete comments. The pluck method should throw an IllegalArgumentException if the key is not one of the 37 keys it is designed to play (as noted above, this differs from the playNote method that simply ignores notes it cant play). Recall that strings have an indexOf method that you might find helpful.

As noted in the description of the interface, the method called time is optional. It is not implemented in the

GuitarLite class, but you should implement it in the Guitar37 class.

In addition to generalizing the code, you will also add the following feature to Guitar37.java to reinforce your understanding of Java's map interface and understand how users interact with your instrument.

- Track how many times each key is plucked
- Write a method to retrieve and display that data (Draw an ASCII histogram as follows):

```
q: ********** (12)
2: ***        (3)
w: *********   (9)
```

    - Implementation Hints:
        - Create a Map <Character, Integer> frequency map
        - In your pluck method, increment the count in the map
- Be sure to test your histogram in GuitarHero

In terms of correctness, your class must provide all of the functionality described above and must satisfy all of the constraints mentioned in this writeup. In terms of style, we will be grading on your use of comments, good variable names, consistent indentation, minimal fields and good coding style to implement these operations.

You should turn the following two on Canvas: GuitarHero.java and Guitar37.java